

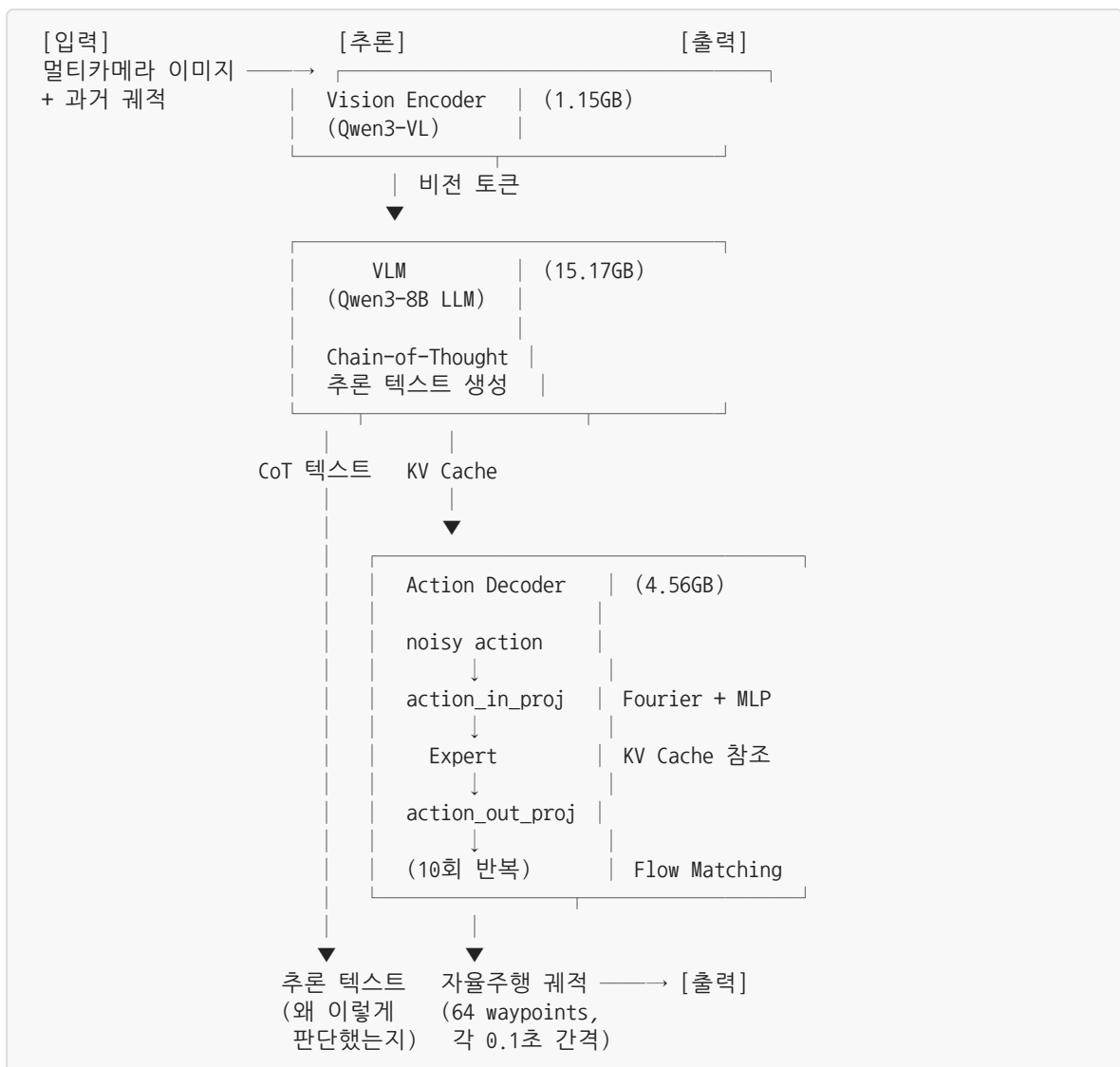
목차 (Table of Contents)

1. 전체 파이프라인 개요
2. 각 모듈 상세
 - 2-1. Vision Encoder
 - 2-2. VLM (Vision-Language Model)
 - 2-3. Action Decoder (Expert + Flow Matching)
3. Action 공간
4. 모듈별 크기 요약
5. 데이터 흐름 (텐서 형태)
6. fuse_traj_tokens() 동작
7. 메모리 최적화 관점에서의 아키텍처 분석
 - 시간 순서에 따른 모듈 활성화
 - 최적화 포인트

Alpamayo-R1-10B 아키텍처 정리

작성일: 2026-02-25

1. 전체 파이프라인 개요



2. 각 모듈 상세

2-1. Vision Encoder

항목	내용
기본 모델	Qwen3-VL의 Vision 부분
크기	~1.15 GB (BF16)
입력	멀티카메라 이미지 (전방, 측면 등)
출력	비전 토큰 (이미지를 패치 단위로 분해한 임베딩)
역할	이미지를 VLM이 이해할 수 있는 토큰으로 변환

2-2. VLM (Vision-Language Model)

항목	내용
기본 모델	Qwen3-VL-8B-Instruct의 LLM 부분
크기	~15.17 GB (BF16)
레이어 수	36 Transformer 레이어
Hidden size	4,096
Attention heads	32
입력	비전 토큰 + 과거 궤적 토큰 + 텍스트 프롬프트
출력	Chain-of-Thought 추론 텍스트 + KV Cache

VLM이 하는 일: 1. 비전 토큰(이미지 정보)과 과거 궤적 토큰을 입력받음 2. 자동회귀적으로 텍스트 생성 (최대 256토큰) 3. “전방에 보행자가 있어 감속해야 한다” 같은 추론(CoT) 생성 4. 이 과정에서 축적된 **KV Cache**를 Expert에 전달

특수 토큰 구조:

```
<|traj_history_start|> 과거궤적토큰x48 <|traj_history_end|>  
... (프롬프트) ...  
<|cot_start|> 추론 텍스트 생성 <|cot_end|>  
<|traj_future_start|> 여기서 VLM 생성 종료, Expert로 전환
```

2-3. Action Decoder (Expert + Flow Matching)

전체 크기: ~4.56 GB (BF16)

action_in_proj (입력 변환)

항목	내용
구조	Fourier Encoding + 4층 MLP
입력	noisy action (B, 64, 2) + timestep
출력	Expert 임베딩 (B, 64, 4096)
파라미터	~6.5M

```

noisy action (64, 2)      timestep
  ↓ Fourier(dim=20)      ↓ Fourier(dim=20)
(64, 40)                (20)
  ↓ concat → (64, 60)
  ↓ MLP: 60 → 1024 → 1024 → 1024 → 4096
  ↓ LayerNorm
임베딩 (64, 4096)
  
```

Expert (핵심 트랜스포머)

항목	내용
구조	VLM의 text_config를 복제한 별도 트랜스포머
레이어 수	32
Hidden size	4,096
Attention	Non-causal (양방향)
입력	action 임베딩 (64, 4096) + VLM의 KV Cache
출력	hidden state (64, 4096)

VLM과의 핵심 차이: - VLM: causal attention (왼→오 단방향) - Expert: **non-causal attention** (양방향) — 64개 waypoint 전체를 동시에 참조

action_out_proj (출력 변환)

항목	내용
구조	Linear(4096, 2)
입력	Expert 출력 (64, 4096)
출력	velocity 벡터 (64, 2)

Flow Matching (반복 적분)

항목	내용
방법	ODE 기반 Euler 적분
스텝 수	10 (기본값)
시간 구간	t: 0.0 → 1.0

```
x = random noise (64, 2)

for i in 0..9:
    v = Expert가 예측한 벡터 필드    ← Expert 1회 호출
    x = x + 0.1 × v                ← Euler 적분

→ x = 최종 action (64, 2)
```

3. Action 공간

Expert가 출력하는 action (64, 2)의 의미:

차원	물리량	범위
dim 0	acceleration (가속도)	-9.8 ~ 9.8 m/s ²
dim 1	curvature (곡률)	-0.2 ~ 0.2 1/m

- 64개 = **6.4초 후까지의 미래** (0.1초 간격 × 64 waypoints)
- 이 action은 `action_to_traj()` 로 실제 3D 궤적 (xyz + 회전행렬)으로 변환됨

Action → 궤적 변환 과정:

acceleration → 적분 → velocity → 적분 → position (x, y)
curvature × velocity → 적분 → heading (theta) → rotation matrix

4. 모듈별 크기 요약

모듈	파라미터	크기 (BF16)	비율
Vision Encoder	~0.6B	1.15 GB	5.5%
VLM (LLM)	~8.2B	15.17 GB	72.6%
Expert	~2.3B	4.26 GB	20.4%
action_in_proj	~6.5M	0.01 GB	0.1%
action_out_proj	~8K	<0.01 GB	~0%
Flow Matching	0	0 GB	0%
합계	~11.08B	~20.9 GB	100%

핵심: VLM이 전체의 72.6%를 차지 → **VRAM 최적화의 핵심 타겟**

5. 데이터 흐름 (텐서 형태)

[입력]

이미지: $(B, n_cams, H, W, 3)$

과거 궤적: $xyz (B, 1, T_hist, 3), rot (B, 1, T_hist, 3, 3)$

↓ Vision Encoder

비전 토큰: $(B, n_patches, 4096)$

↓ 과거 궤적 토큰화 (DeltaTrajectoryTokenizer)

궤적 토큰: $(B, 48) \leftarrow$ 정수 인덱스 (0~999)

↓ fuse_traj_tokens() — 궤적 토큰을 input_ids에 삽입

input_ids: $(B, seq_len) \leftarrow$ 비전 + 궤적 + 프롬프트 통합

↓ VLM.generate() — 자동회귀 텍스트 생성

생성된 시퀀스: $(B \times 6, new_seq_len) \leftarrow num_traj_samples=6$

KV Cache: $(32\ layers, 2, B \times 6, seq_len, 4096)$

↓ Flow Matching 시작

초기 noise: $(B \times 6, 64, 2) \sim N(0,1)$

↓ 10 스텝 반복:

↓ action_in_proj

임베딩: $(B \times 6, 64, 4096)$

↓ Expert (KV Cache attention)

hidden: $(B \times 6, 64, 4096)$

↓ action_out_proj

벡터 필드: $(B \times 6, 64, 2)$

↓ Euler: $x = x + dt \times v$

최종 action: $(B \times 6, 64, 2)$

↓ action_to_traj() — action $\rightarrow$ 3D 궤적 변환

[출력]

pred_xyz: $(B, 1, 6, 64, 3) \leftarrow$ 6개 후보 궤적

pred_rot: $(B, 1, 6, 64, 3, 3)$

CoT 텍스트: 추론 근거

6. fuse_traj_tokens() 동작

device_map="auto"가 Alpamayo에서 실패하는 원인이 되는 함수.

```
def fuse_traj_tokens(input_ids, traj_data):
    # 1. 과거 궤적을 discrete 토큰으로 변환
    #   DeltaTrajectoryTokenizer: xyz 좌표 → 델타 → 양자화 → 정수 인덱스
    hist_idx = tokenize_history_trajectory(...) # (B, 48)

    # 2. input_ids에서 패딩 위치(<|traj_history|>)를 찾아 궤적 토큰으로 교체
    #   masked_scatter 사용 → GPU/CPU 텐서 혼합 시 에러 발생
    input_ids = replace_pad_token(input_ids, hist_idx, pad_idx=...)

    return input_ids
```

device_map 실패 원인: `masked_scatter` 연산 시 `input_ids`와 `hist_idx`가 서로 다른 디바이스 (GPU/CPU)에 있으면 CUDA assertion error 발생

7. 메모리 최적화 관점에서의 아키텍처 분석

시간 순서에 따른 모듈 활성화

시간 →

Phase 1: [Vision Encoder]  (1.15GB, 짧음)

Phase 2: [VLM 생성]  (15.17GB, 길음)

Phase 3: [Expert  10]  (4.56GB, 중간)

- Phase 1~30이 순차적 → 한 번에 하나의 모듈만 활발히 사용
- 그러나 VLM의 KV Cache가 Phase 3에서도 필요 → 완전한 언로드 불가
- VLM 단독(15.17GB) > 12GB VRAM → VLM 내부에서도 레이어 스와핑 필요

최적화 포인트

포인트	설명	절감 가능
Phase 순차 실행	Vision/VLM/Expert를 순서대로 로드/언로드	~6GB
VLM 레이어 스와핑	36개 레이어를 12GB 내에서 순차 실행	필수
KV Cache 관리	VLM 생성 후 Expert 전용 캐시만 유지	~1-2GB
Expert 최적화	Flow Matching 스텝 수 조절 (10→5)	추론 시간 절반